

NI1 – Programação Orientada a Objetos: Documentação do Jogo Space Invaders

Turma: 2NAADS

Data da apresentação: 23/10/2026

Integrantes: [nome 1], [nome 2], [nome 3], [nome 4]

Tecnologia: JavaScript com `<canvas>` (arquivo único HTML)

1. Visão geral

O grupo desenvolveu um jogo no estilo Space Invaders. O jogador controla uma nave na parte de baixo da tela e deve destruir ondas de inimigos que descem em formação. O jogo termina em vitória ao chegar a 1000 pontos, ou em derrota se as vidas acabarem ou se os inimigos alcançarem a nave.

Tecla	Ação
Setas ou A/D	Mover a nave
Espaço	Atirar
P ou Esc (ou botão)	Pausar e continuar
Enter	Iniciar ou jogar de novo

[inserir print do menu] [inserir print da partida]

2. Conceitos de POO aplicados

2.1 Hierarquia de classes

```
Entidade
├── Projétil
├── Jogador
├── PowerUp
├── Inimigo
│   ├── InimigoBasico
│   ├── InimigoResistente
│   └── InimigoAtirador
```

2.2 Classes e atributos

Classe	Herda de	Atributos principais	Função
Entidade	-	x, y, largura, altura, cor, vivo	Base de todos os objetos; tem desenhar(), atualizar() e colideCom()
Projétil	Entidade	velocidadeY	Tiro do jogador (sobe) e dos inimigos (desce)
Jogador	Entidade	velocidade, vidas, recarga, invulneravel	Nave controlada pelo usuário
PowerUp	Entidade	velocidade	Cai de inimigos destruídos e dá 1 vida
Inimigo	Entidade	pontos, vida	Comportamento padrão dos inimigos
InimigoBasico	Inimigo	-	Vermelho, 10 pontos, 1 tiro para destruir, não atira
InimigoResistente	Inimigo	-	Amarelo, 20 pontos, aguenta 2 tiros
InimigoAtirador	Inimigo	-	Roxo, 30 pontos, atira no jogador

2.3 Herança

Jogador, Projétil, PowerUp e Inimigo herdam de Entidade e reaproveitam posição, tamanho e colisão por meio de `super(...)`. Há dois níveis de herança: Entidade → Inimigo → três tipos de inimigo.

2.4 Polimorfismo (sobrescrita de métodos)

A classe Inimigo define comportamentos padrão, e cada filha os sobrescreve quando precisa:

```
class Inimigo extends Entidade {
    tentarAtirar() { return null; }    // padrão: não atira
}

class InimigoAtirador extends Inimigo {
    tentarAtirar() {                    // sobrescreve: devolve um projétil
        return new Projétil(this.x + this.largura / 2 - 2, this.y +
this.altura, 5, '#c266ff');
    }
}
```

O jogo sorteia um inimigo e chama `sorteado.tentarAtirar()` sem saber o tipo dele: o básico devolve `null` e o atirador devolve um `Projétil`. O mesmo vale para `desenhar()`, que é diferente em cada tipo de inimigo, e para `levarTiro()`, que muda no inimigo resistente.

2.5 Encapsulamento

Cada classe controla o próprio estado. Por exemplo, o jogo chama `jogador.levarDano()` em vez de alterar `vidas` e `invulneravel` por fora.

3. Ferramentas de IA utilizadas

IA	Uso no projeto
Gemini	Gerou o código-base "Sobrevivência Espacial" (desviar de blocos)
Claude	Evoluiu a base para um Space Invaders completo, com tiros, ondas e novas classes

Observação: o Space Invaders do Claude foi construído a partir do código do Gemini. Portanto, esta comparação mostra a base do Gemini contra a versão evoluída pelo Claude, e não duas IAs recebendo o mesmo pedido.

4. Diário de desenvolvimento

Etapa 1 – Código-base (Gemini)

- **Prompt usado:** [colar aqui o prompt exato enviado ao Gemini]
- **Resultado:** jogo "Sobrevivência Espacial": a nave desvia de blocos vermelhos e apanha blocos verdes, com vitória aos 500 pontos.

- **Classes geradas:** Entidade , Jogador , Inimigo , PowerUp .
- **Evidência:** [inserir print]

Etapa 2 – Criação do Space Invaders (Claude)

- **Prompt usado:** "crie um jogo space-invaders a partir deste código" (com o código do Gemini anexado)
- **Resultado:** jogo com formação de inimigos, tiros, vidas, ondas, power-up e 3 tipos de inimigo.
- **Evidência:** [inserir print e GIF da partida]

Etapa 3 – Botão de pausa (Claude)

- **Prompt usado:** "coloque um botão de pausa no jogo"
- **Resultado:** botão "Pausar/Continuar", teclas P e Esc, tela "PAUSADO" e pausa automática ao trocar de aba.
- **Como funciona:** alternarPausa() só muda o estado entre 'jogando' e 'pausado' ; o loop principal só atualiza a lógica no estado 'jogando' .
- **Evidência:** [inserir print]

5. Bugs encontrados e correções

Bug 1 – CSS sem seletor (código do Gemini)

- **Descrição:** as regras margin , padding e box-sizing estavam soltas dentro do <style> , sem seletor, e por isso eram ignoradas pelo navegador.
- **Antes:**

```
margin: 0;
padding: 0;
box-sizing: border-box;
```

- **Depois:**

```
* { margin: 0; padding: 0; box-sizing: border-box; }
```

Bug 2 – Tela de "Game Over" não some ao reiniciar (código do Gemini)

- **Descrição:** em reiniciarJogo() , a classe escondido era adicionada e logo em seguida overlay.className = '' a apagava. O jogo reiniciava por baixo, mas a tela de fim continuava visível.

• **Antes:**

```
overlay.classList.add('escondido');
overlay.className = '';
```

• **Depois:**

```
overlay.className = 'escondido';
```

Bug 3 – Fim de jogo podia ser chamado duas vezes (código do Gemini)

- **Descrição:** o loop não parava depois de `fimDeJogo(false)` . Se no mesmo quadro a pontuação chegasse a 500, `fimDeJogo(true)` também era chamado e mostrava "VITÓRIA" depois de uma derrota.
- **Correção na versão do Claude:** o fim de jogo é decidido com `if / else if` , então só um dos casos acontece por quadro.

Bug 4 – CSS sem uso (código do Gemini)

- **Descrição:** as regras `.inimigo.basico` , `.inimigo.rapido` e `.inimigo.atirador` não eram aplicadas a nenhum elemento.
- **Correção:** os tipos de inimigo passaram a existir de verdade como classes (`InimigoBasico` , `InimigoResistente` , `InimigoAtirador`) e as regras sem uso foram removidas.

Bugs encontrados pelo grupo nos testes

[preencher com os bugs que o grupo achar ao jogar: descrição, código antes e depois, print]

6. Comparação entre as IAs

Critério	Gemini (Sobrevivência Espacial)	Claude (Space Invaders)
Tipo de jogo	Desviar de blocos que caem; sem tiro	Space Invaders: formação, tiros, ondas, vidas
Classes	4	8
Herança	Um nível	Dois níveis

Critério	Gemini (Sobrevivência Espacial)	Claude (Space Invaders)
Polimorfismo	Fraco: <code>atualizar()</code> repetido e <code>desenhar()</code> nunca sobrescrito	Mais claro: <code>tentarAtirar()</code> , <code>levarTiro()</code> e <code>desenhar()</code> sobrescritos
Comentários	Poucos no JavaScript	Comentários nas classes e nos blocos principais
Velocidade	Arelada aos quadros por segundo	Escala de tempo; mesma velocidade em qualquer monitor
Layout	Largura fixa	Se adapta à tela, com botões de toque
Simplicidade	Mais curto e fácil de explicar	Mais longo, exige mais estudo do grupo

Conclusão

O código do Gemini foi uma base simples e organizada, boa para começar, mas cobria o mínimo de POO e continha bugs. A versão do Claude ampliou a hierarquia e deixou o polimorfismo mais evidente. Como contrapartida, é maior e mais complexa, e a lógica do jogo está em funções globais, e não em uma classe `Jogo`. [o grupo pode acrescentar aqui a própria avaliação após testar os dois]

Limitação: a análise dos códigos foi feita pela leitura do programa. O grupo deve testar os dois jogos e registrar os resultados.

7. Contribuição de cada integrante

Integrante	O que fez	Classes ou partes que domina
[nome 1]	[descrever]	[descrever]
[nome 2]	[descrever]	[descrever]
[nome 3]	[descrever]	[descrever]
[nome 4]	[descrever]	[descrever]

8. Créditos e referências

- Código-base gerado pelo Gemini e evoluído com o Claude, conforme descrito nas seções 3 e 4.
- Não foram usados assets nem tutoriais de terceiros: os gráficos são desenhados por código.
- [acrescentar qualquer outra fonte usada pelo grupo]